HCL AppScan Local MCP Server

Client Compatibility, Functional Validation & End-to-End SAST Test Report

Report date: 5 November 2026
Package: mcp-appscan-server@1.0.0
Clients: Claude Desktop and Claude Code
Runtime: Node.js 22.23.4

Overall result: PASS — all planned test areas recorded in the test tracker passed, subject to the observations and limitations in this report.

1. Executive summary

The standalone local HCL AppScan MCP server (version 1.0.0) was validated from both Claude Desktop and Claude Code. Coverage included MCP connectivity, server information, tool discovery and schemas, read-only AppScan operations, input validation, approval workflows, application creation, SAST scan execution, and cross-client verification. The key end-to-end workflow succeeded: an application was created through a preview/confirmation-token flow; a source-code-only SAST scan was previewed, explicitly approved, uploaded and completed; and the resulting scan and its four findings were independently verified across both clients.

2. Scope and test environment

• Server package: mcp-appscan-server@1.0.0.
• Runtime: Node.js 22.23.4; package requirement recorded as Node.js >=20.0.0 and <23.
• Clients: Claude Desktop and Claude Code; local STDIO server.
• AppScan tenant context: HCL Plugins Team.
• Coverage: connection, metadata, tool inventory/schemas, tenant/application/scan/issue/fix-group/asset-group/report reads, input validation, approval workflows, application creation, SAST execution and cross-client comparison.

3. Test results summary

All 23 top-level test areas in the tracker were recorded as PASS. TC-10/10A, TC-12/12A and TC-13/13A include definition and execution subchecks. TC-21 to TC-23 cover the consequential application/scan workflow and verification.

Test ID	Test area	Result	Evidence / observation
TC-01	MCP connection	PASS	Both clients connected.
TC-02	get-server-info	PASS	Matching server version, API base, credential-configured flag, timeout, retry count and log level.
TC-03	Tool discovery	PASS	Same 14 tools discovered.
TC-04	Input schema validation	PASS	Core schemas consistent; minor presentation/default interpretation differences.
TC-05	get-tenant-info	PASS	Matching tenant and current-user information.
TC-06	get-apps	PASS	Same 22 application records.
TC-07	get-scans	PASS*	Both returned 71 scans; Claude Code had some malformed/truncated UUID display.
TC-08	get-issues	PASS	Same four issues for selected application.
TC-09	get-scan-issues (existing scan)	PASS	Same four findings for selected existing scan.
TC-10/10A	get-fix-groups	PASS	Consistent definitions; same four fix groups returned.
TC-11	get-issue-details	PASS	Matching details for selected SQL injection issue.
TC-12/12A	get-asset-groups	PASS	Same single asset group.
TC-13/13A	get-reports	PASS	Consistent definitions; both returned an empty report list.
TC-14	create-app definition	PASS	Approval behavior reviewed; runtime tested in TC-21.
TC-15	create-scan definition	PASS	Parameters and approval behavior reviewed; runtime tested in TC-22.
TC-16	update-fix-groups definition	PASS	Matching write-tool definitions and parameters.
TC-17	update-issues definition	PASS	Matching write-tool definitions and parameters.
TC-18	Tool inventory	PASS	Same 14 tools; no delete tool discovered.
TC-19	Invalid app ID	PASS	Malformed UUID rejected by API with HTTP 400; no data changed.
TC-20	Missing app ID	PASS	MCP -32602 input validation error; no API operation.
TC-21	Application creation	PASS	Claude Desktop created via preview/token; Claude Code verified same ID; no duplicate.
TC-22	SAST scan workflow	PASS	Preview, explicit approval, IRX upload, scan completion and Claude Code verification.
TC-23	Findings comparison	PASS	Identical four findings across clients: IDs, titles, severity, status and locations.

4. Application creation workflow

Application: MCP_POC_Approval_Test
Application ID: 50588b82-e61e-4bd9-b61b-cc455b045362

Claude Desktop performed the preview/confirmation-token flow and created the application after explicit user approval. Claude Code did not create a duplicate; it performed a read-only get-apps lookup and verified exactly one matching application with the same ID. The user independently confirmed the application was visible in the AppScan console. The application is a persistent tenant-side change. No delete tool was found in the tested 14-tool inventory; cleanup, if required, must use another supported administrative route.

5. End-to-end SAST scan execution

Field	Value
Application	MCP_POC_Approval_Test (50588b82-e61e-4bd9-b61b-cc455b045362)
Scan name	MCP_POC_SAST_Approval_Test
Scan ID	d06055c6-14a1-4b04-81a6-705cdd930f3d
Scan type	SAST (confirmed by create-scan execution response)
Scan mode	source-code-only; no project build executed
Scan level	normal
Status	Ready; progress 100%
Duration	23 seconds
Remote IRX file ID	169b4562-4f4a-4909-8a21-06001e6a1c20
Findings	4 total: 2 High, 1 Medium, 1 Low
Errors / timeout	None reported; completed well within configured 120-minute wait
Local cleanup	Operation folder and IRX reported deleted because conserveFiles=false. Generated config cleanup was expected by preview but not explicitly confirmed in completion response.

Approval sequence: Claude Desktop first returned a preview and confirmation token. No upload or scan occurred at preview time. After explicit user approval, the token was accepted and the scan completed. The token is intentionally omitted from this report.

Upload evidence: the completion response reported a remote IRX file ID and stated that the scan ran on the uploaded artifact. It did not use the literal phrase “upload succeeded”; successful scan completion against the reported remote artifact is strong corroborating evidence.

Cross-client verification: Claude Code called get-scans read-only and found exactly one match for the scan ID, with expected name, application ID and Ready status at 100%. get-scans did not return scan type, issue counts, creation date or risk in that response.

6. Findings returned for the new scan

Both clients returned the same four issue records. Paths are shortened for readability; the full project-relative prefix was spring-boot-restful-web-service-example-master\src\main\java\com\levent\.

Issue	Issue ID	Severity	Status	Location
Cross-Site Scripting	96b4fd73-00d9-f111-bf5b-000d3ae5cac2	High	Open	...\config\XSSVulnerableServlet.java:22
Configuration	99b4fd73-00d9-f111-bf5b-000d3ae5cac2	Low	Open	...\controller\ConsultantController.java:29
Improper Handling of Exceptional Conditions	9cb4fd73-00d9-f111-bf5b-000d3ae5cac2	Medium	Open	...\controller\ConsultantController.java:42
SQL Injection	9fb4fd73-00d9-f111-bf5b-000d3ae5cac2	High	Open	...\service\consultant\impl\ConsultantServiceImpl.java:60

Severity reconciliation: Critical 0, High 2, Medium 1, Low 1, Informational 0. These counts match the scan-completion summary.

Pagination qualification: neither client response supplied a total count, page token or completeness flag. Four returned issues are below the default limit of 100, so truncation is unlikely, but completeness is not independently confirmed by pagination metadata.

7. Observations, limitations and recommendations

- **Output formatting:** Claude Code rendered some long tables and scan UUIDs poorly/truncated in earlier get-scans output. Improve structured output formatting and preserve UUIDs.
- **Scan metadata:** get-scans did not return scan type, creation date, risk or issue counts in the tested record. Expose supported fields or label them unavailable.
- **Pagination transparency:** get-scan-issues did not expose pagination or an explicit completeness indicator. Return total count, paging metadata and/or a result-complete flag.
- **Input validation:** Missing appId was rejected at MCP layer (-32602), while malformed UUID reached AppScan API and returned HTTP 400. Add UUID-format validation at the MCP boundary.
- **Tool classification:** 14 tools were identified. Read/write classification was inferred from descriptions. get-reports is mixed: list is read-only, create generates an AppScan report, download writes locally. Write tools include create-app, create-scan, update-fix-groups and update-issues. No delete tool was found.
- **Approval flow:** Tested create-app and create-scan used preview/confirmation-token flows. Preserve explicit approval and ensure tokens bind to exact request parameters and expire appropriately.
- **Credential hygiene:** An AppScan configuration excerpt containing credentials was pasted during setup troubleshooting. Rotate/revoke any credentials exposed in that excerpt and update client configurations with newly issued secrets. Do not put secrets in reports or chat.
- **Persistent test data:** The test application and scan remain in the AppScan tenant. No delete tool was found. Remove only through an approved administrative workflow if cleanup is required.

- **Report listing:** get-reports list returned no reports in the tested context. This does not establish that the entire tenant has no reports.
- **Application count difference:** get-tenant-info reported 29 apps while get-apps returned 22. The difference was observed but not explained.
- **Write retry behavior:** Read retry count was reported as 2. Write idempotency/retry behavior was not exhaustively stress-tested; do not assume writes are safe to replay after ambiguous timeouts.

8. Security and test-safety notes

- The SAST scan used source-code-only mode; no project build ran.
- The scan was explicitly approved before source upload and remote execution.
- update-issues and update-fix-groups were not executed; only their definitions/schemas were reviewed.
- This report excludes API key IDs, secrets, confirmation tokens and other credentials.
- scanSecrets=true was used. Review the scan results and handle any detected secrets under the organization's security process.

9. Conclusion

The local AppScan MCP server passed the planned functional and cross-client validation recorded in this report. The approval-controlled SAST workflow completed successfully from Claude Desktop and was independently verified from Claude Code, with identical four-issue results from both clients. Remaining items are product-quality recommendations and metadata/observability limitations rather than blockers to the tested workflow. This report reflects the evidence shared during the session; it is not a formal security certification or a guarantee that untested workflows are defect-free.

Appendix A — Key identifiers

Application: MCP_POC_Approval_Test — 50588b82-e61e-4bd9-b61b-cc455b045362
Scan: MCP_POC_SAST_Approval_Test — d06055c6-14a1-4b04-81a6-705cdd930f3d
Remote IRX file ID: 169b4562-4f4a-4909-8a21-06001e6a1c20
Package: mcp-appscan-server@1.0.0; Runtime: Node.js 22.23.4